

A Scalable Real-Time Many-Shadowed-Light Rendering System

Bo Li

Warner Bros. Games Montréal



MONTREAL



PHOTOGRAPHY & RECORDING ENCOURAGED

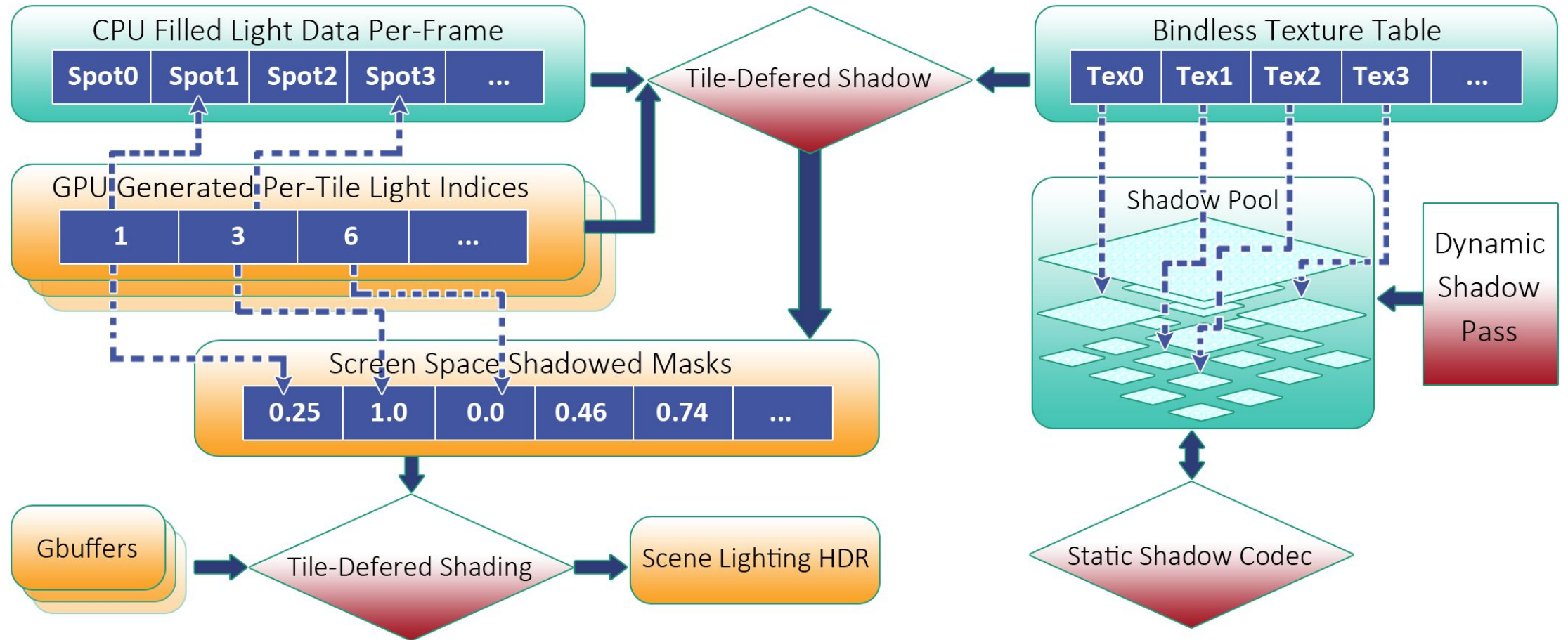


Motivations

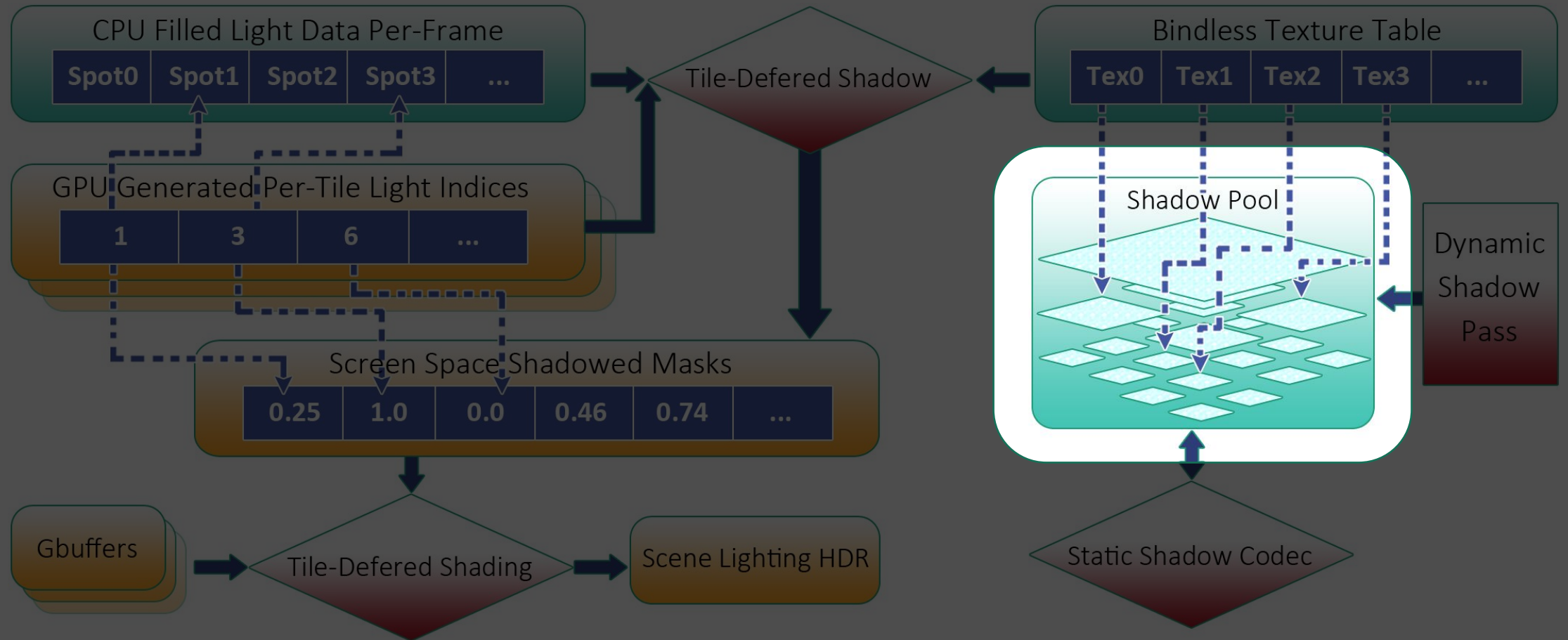
There's no un-shadowed light in the real world

(Unless you are a Quantum Physicist Player)

Our System



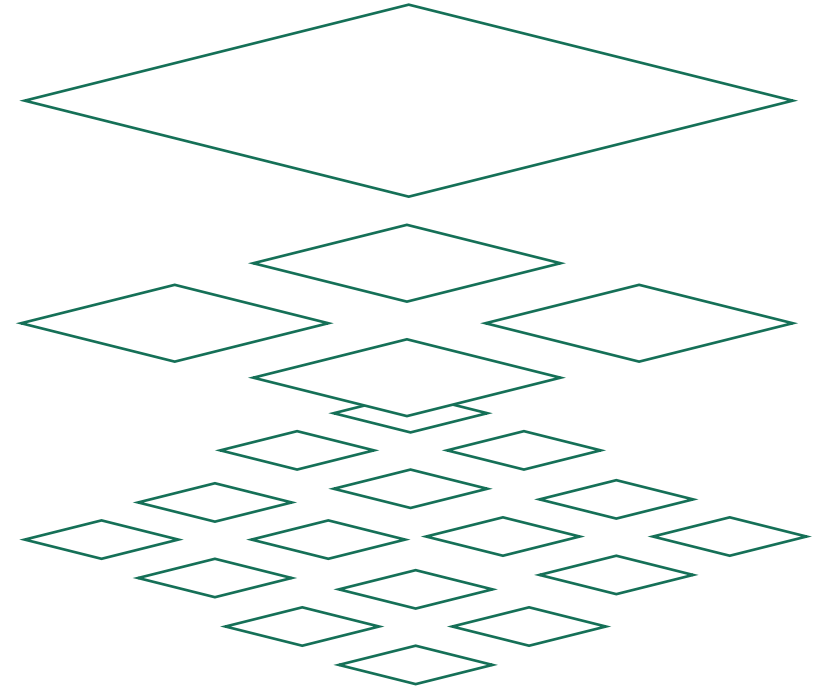
Our System





Multi-Resolution Shadow Map Pool

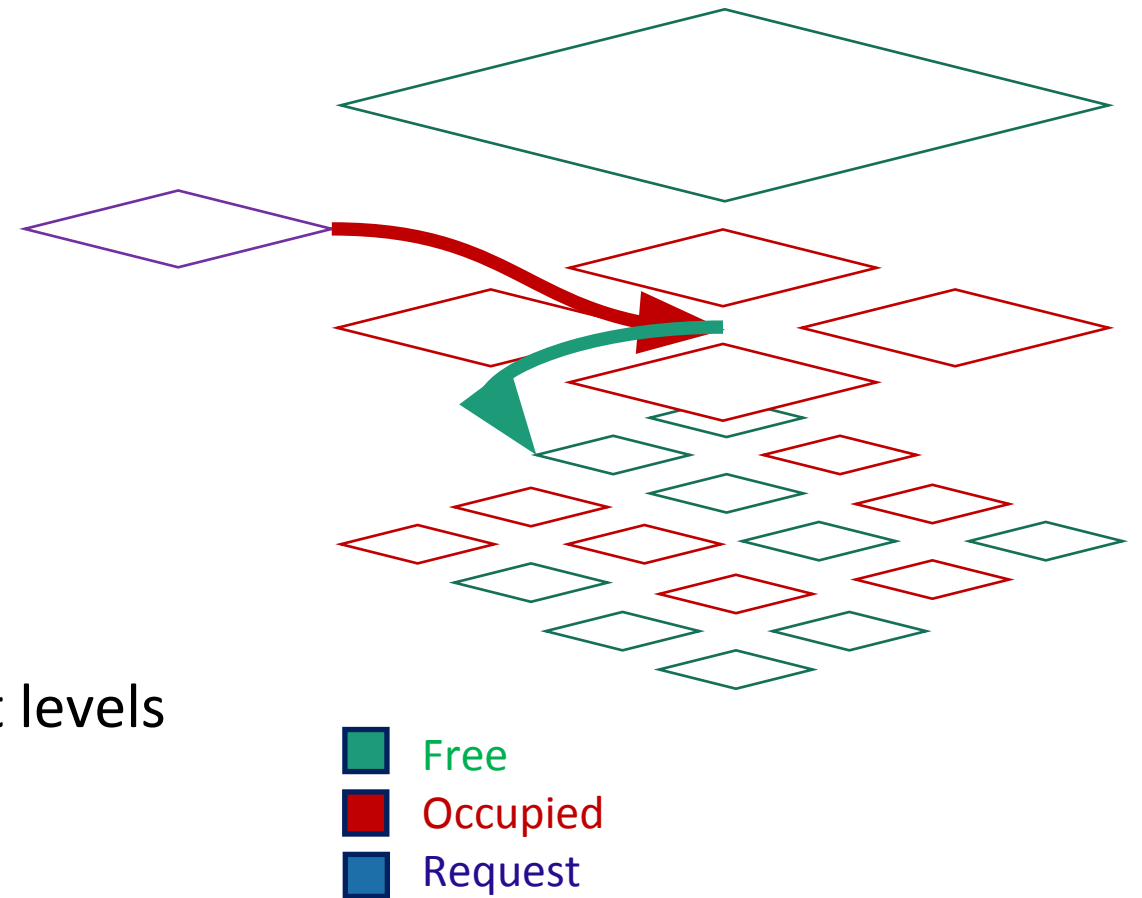
- Lots of **Pre-Allocated** Shadow Textures
- Each subsequent level, **1/4** Resolution and **4x** Number Textures
 - Ideally constant #texels per-level
- Goal: Target constant Pixel/Texel Ratio(1:1)
 - The smaller screen-projected size, the more texture slots available
 - Each Light allocates its best resolution



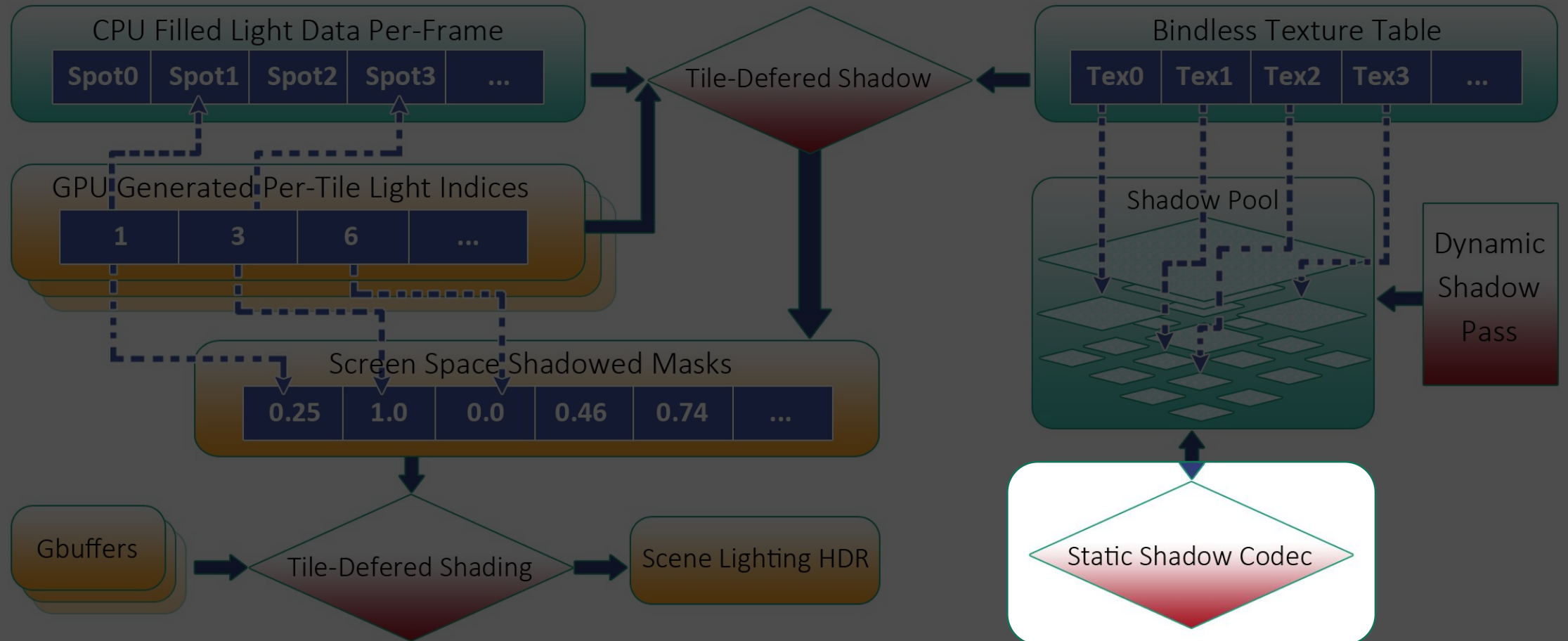


Shadow Pool: In practice

- PlayStation 4 / Xbox One:
 - Max 128 Textures Per-Level
 - First Level: [2048x2048]x4
 - Last Level: [32x32]x128
 - 156 MB, 596 Textures
- Allocation:
 - Search with Max-Desired Resolution
 - If no free texture found, search in next levels



Our System



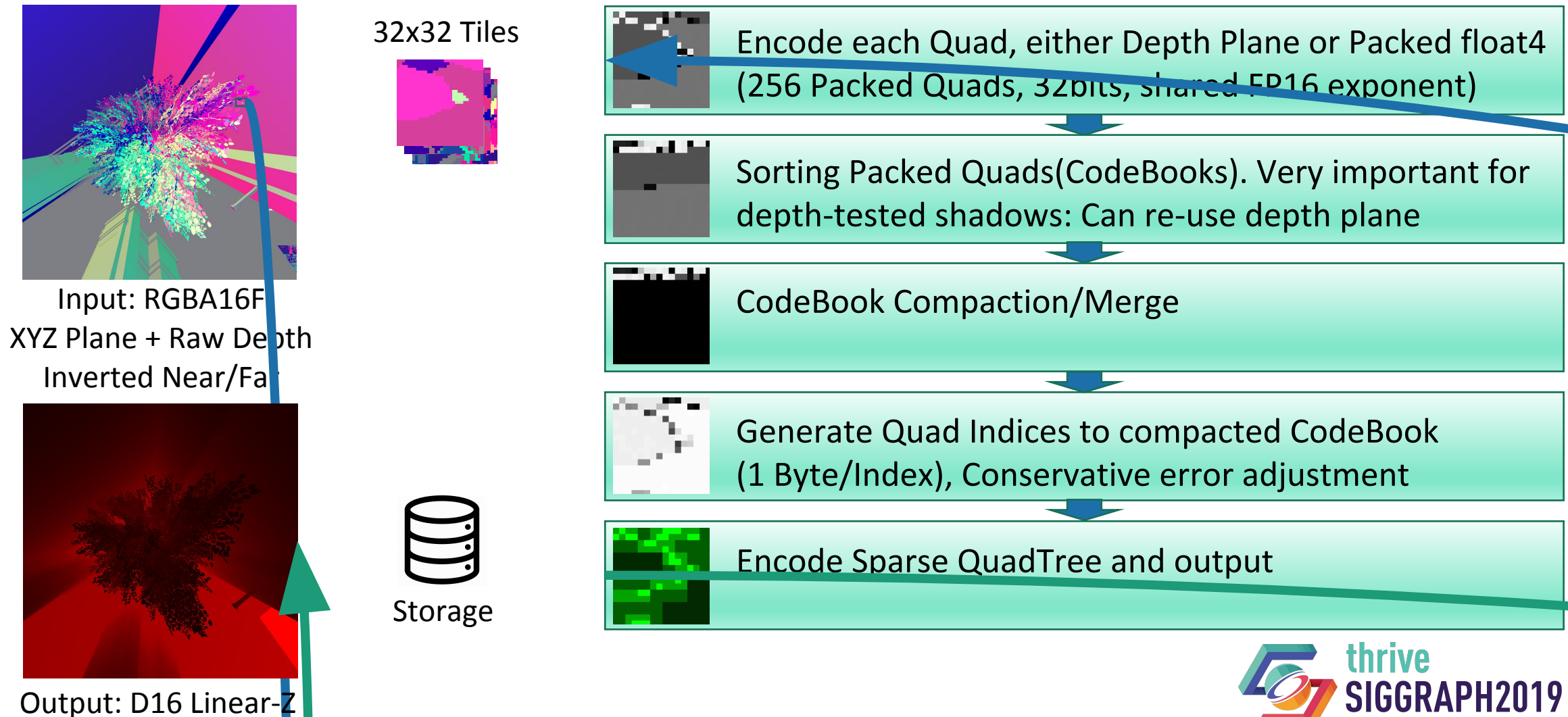


GPU Shadow Map Compression

- Motivations
 - Skip runtime static shadow rendering
 - Parallelism: Overlap Compute/Graphics pipeline
 - Minimize Size
- Challenges
 - CS based compression/decompression: parallel irregular data structure
 - Floating point precision
 - Limited data format on GPU TGSM: 32bit data size
- Lossy but conservative errors (No hand-tweaking Depth/Slope-Bias)



GPU Shadow Map Compression: Data Flow





GPU Shadow Map Compression: Result

- Designed to handle **Alpha-Testing**: Unique depth planes shared
- Compression Ratio:
 - Typically anywhere between **7:1** to **100:1**
 - Worst case **1.45:1** (pure noise input), best case **512:1** (single depth plane/tile)
 - Can expect average **20:1** or better, prefer larger textures
- Decompression Speed:
 - **0.048ms** for 1024x1024 on PS4 Base
 - Close to hardware pixel fill-rate
- Compression Speed:
 - **0.36ms** for 1024x1024 on PS4 Base (unoptimized)
 - Todo: use LaneSwizzle instruction for sorting/scan



Shadow Map Compression: Quality



Original Shadow Map

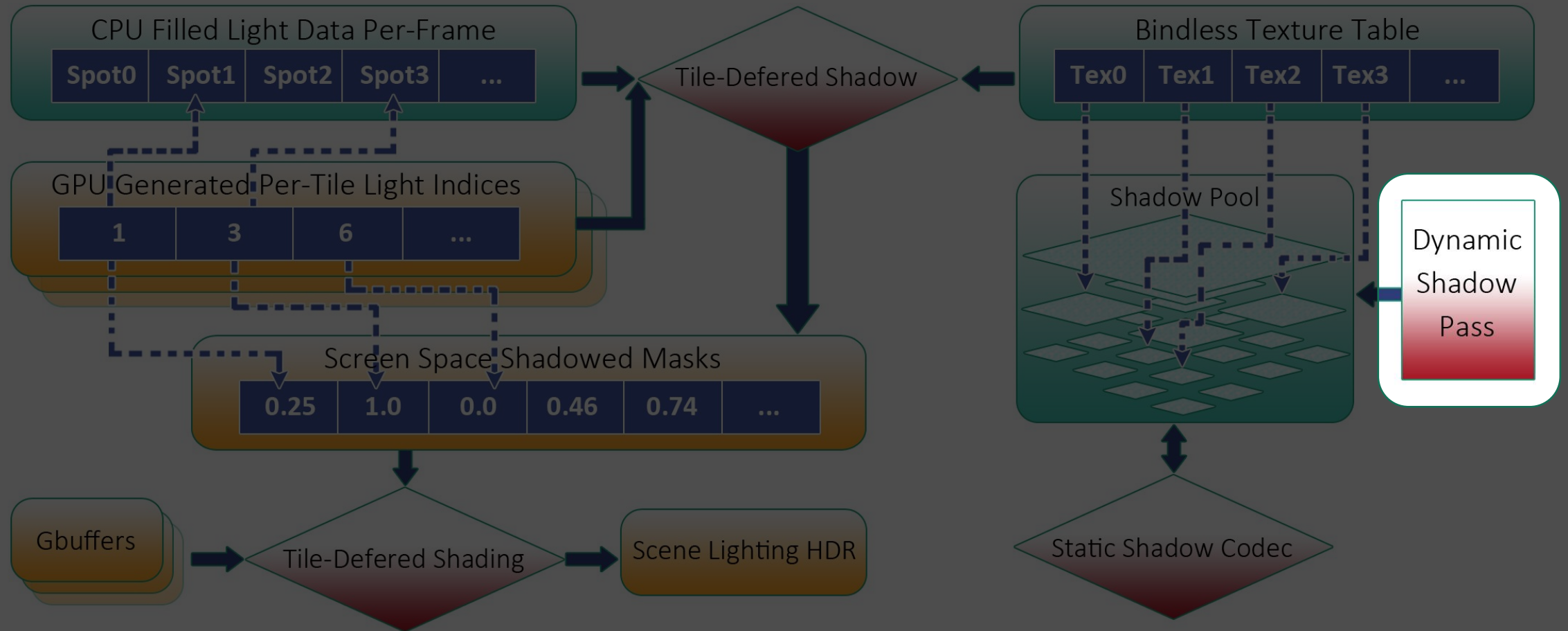


Static Shadow Map Compression: Quality



Compressed Shadow Map(7.15 : 1)

Our System





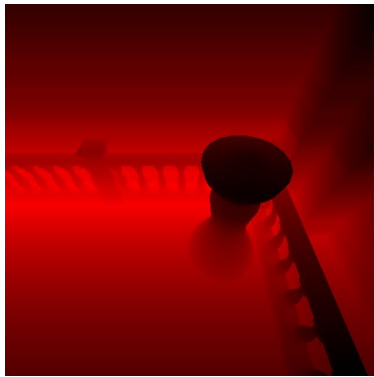
Dynamic Shadow Pass

- Goal: Minimize overhead
 - Full Depth Copy
 - Full Depth Clear / Depth Decompression
- Traditional Options:
 - Full Shadow Copy From Static to Dynamic: Slow, High Fixed Cost
 - Re-Generate Static Shadow: Very high CPU Cost
 - Sample Both Static and Dynamic Shadow Maps: High Filtering Cost
- But Dynamic-Only Shadow is often highly-sparse in texture space
 - Full shadow map copy/merging is undesirable
 - Double filtering cost is undesirable

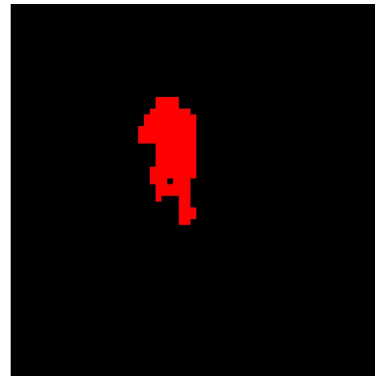


Separated Dynamic Shadow: Example

- A typical shadow map layout with dynamic interactions:

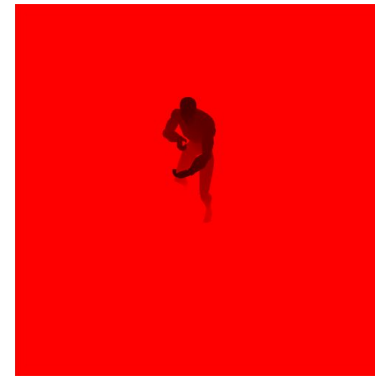


Static
512x512

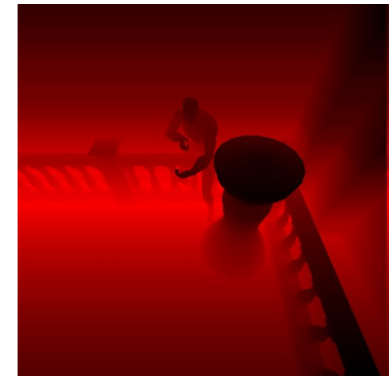


Dynamic-Mask
32x32

=



Dynamic
1024 x 1024

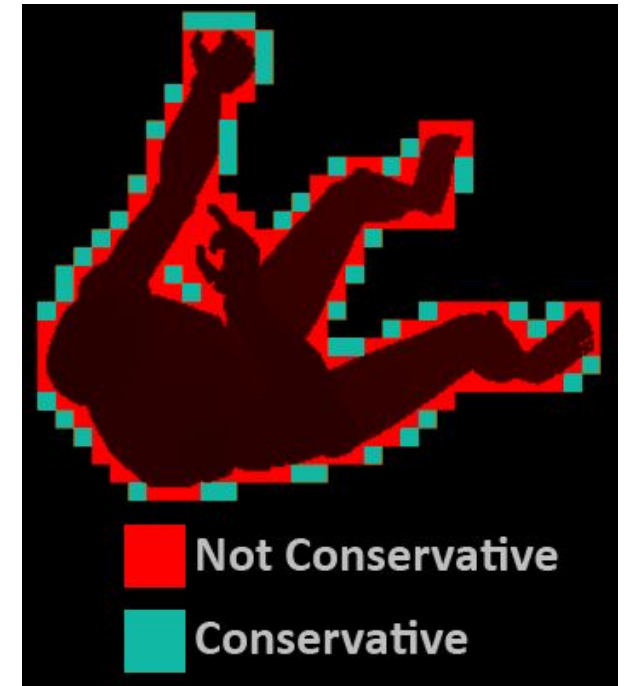


Equivalent



Conservative Dynamic-Mask

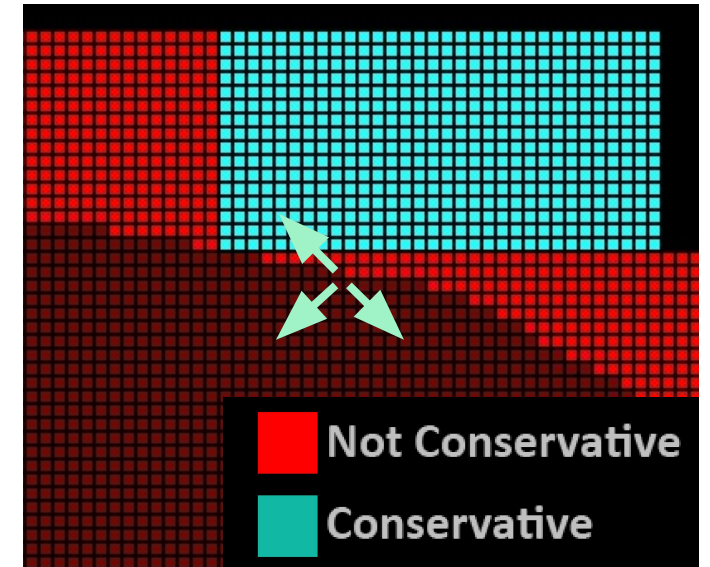
- Filtering: Check Dynamic-Mask Texture **Once** for the **Entire Kernel**
 - Unbound(TextureIndex == -1): Static-Only
 - Texel false: Static-Only
 - Texel true: Dynamic and Static
- Dynamic-Mask must be conservative covering the whole filter kernel





Conservative Dynamic-Mask

- Bound as UAV on Pixel Shader, no AtomicOp needed on Current-Gen Consoles
- **R8_UNORM** On PS4/Xbox, **R32_UINT** On PC (or check “UAV Typed Load” in DX11.3)
- Extrapolate the position from center of a four pixel quad with the shadow filtering kernel radius



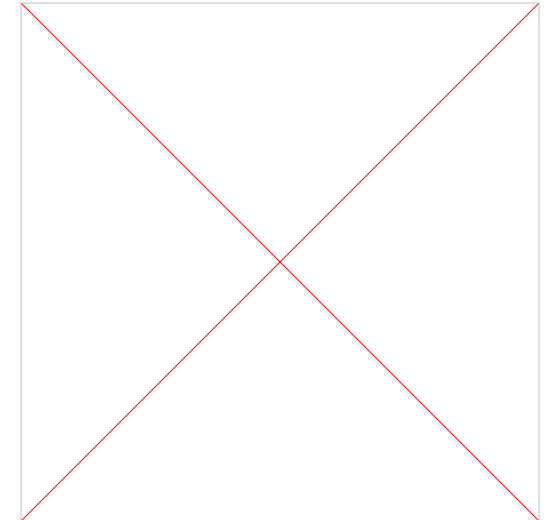
```
float2 ConservativeOffset = (SvPosition.xy & 1) - 0.5f) * Max_Filer_Kernel_Size * 2;  
uint2 LowResCoord = (SvPosition.xy + ConservativeOffset) / 64.f;  
if (ShadowLowResFlags[LowResCoord] == 0) //Avoid some write contentions on some HW  
    ShadowLowResFlags[LowResCoord] = 1;
```



Depth Partial Decompression(PS4/X1)

- Minimized Overhead
 - Only ~0.03ms overhead for 2k x 2k shadow pass(PS4)
 - No full depth copy
 - No “slow” depth clear
 - Partial depth decompression with dynamic mask
- Use **Dynamic-Mask** for partial decompression
 - Generate Rect-List Based on Dynamic-Mask

2048x2048 Depth (Example)	Cost	Variance
Full-Decompress	65.8us	Fixed
Partial-Decompress	9.6us	Data-Dependent





Robust Depth-Bias

- Uniform depth bias will always fail at unbounded depth slope
 - Only used to correct rounding errors
- **SlopeBias = Filter_Kernel_Radius**
 - Geometrically based: (Max Variance Per-Pixel) * Width
 - No User Input Needed
 - HW:
 - `RasterizerDesc.DepthBias = Epsilon; //(1 is a good epsilon choice)`
 - `RasterizerDesc.SlopeScaledDepthBias = Max_Filer_Kernel_Size; //(ex 3.0f)`
 - Note: HW implement `max(ddx(z), ddy(x))`, you might want to use lager value
 - SW:
 - `ShadowDepth += Epsilon / 65535.f; //For R16_Depth`
 - `ShadowDepth += (abs(ddx(ShadowDepth)) + abs(ddy(ShadowDepth))) * Max_Filer_Kernel_Size;`



Robust Depth-Bias: D16

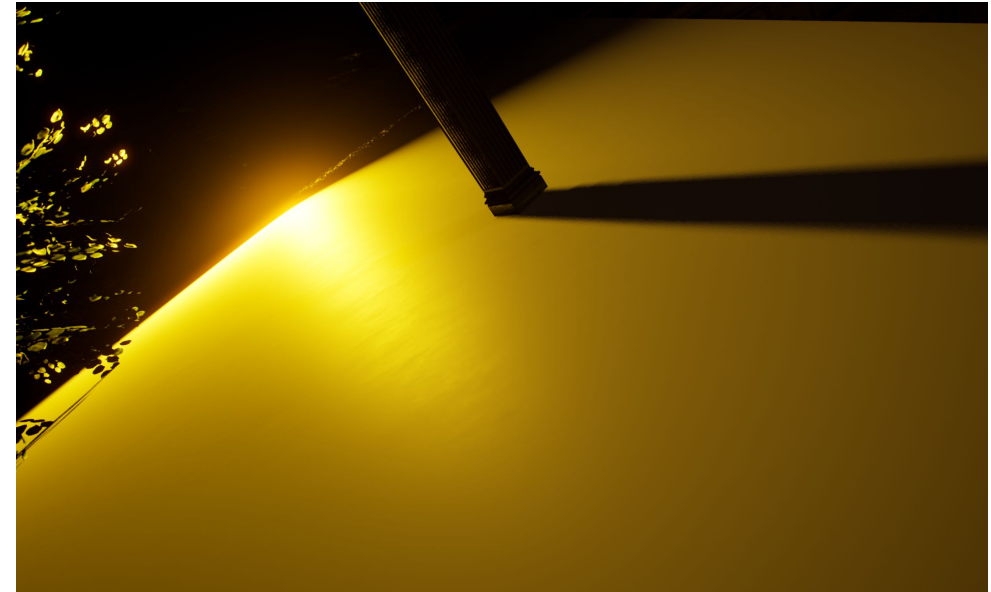
Depth Bias = **0.005**

Slope Bias = **0.0**

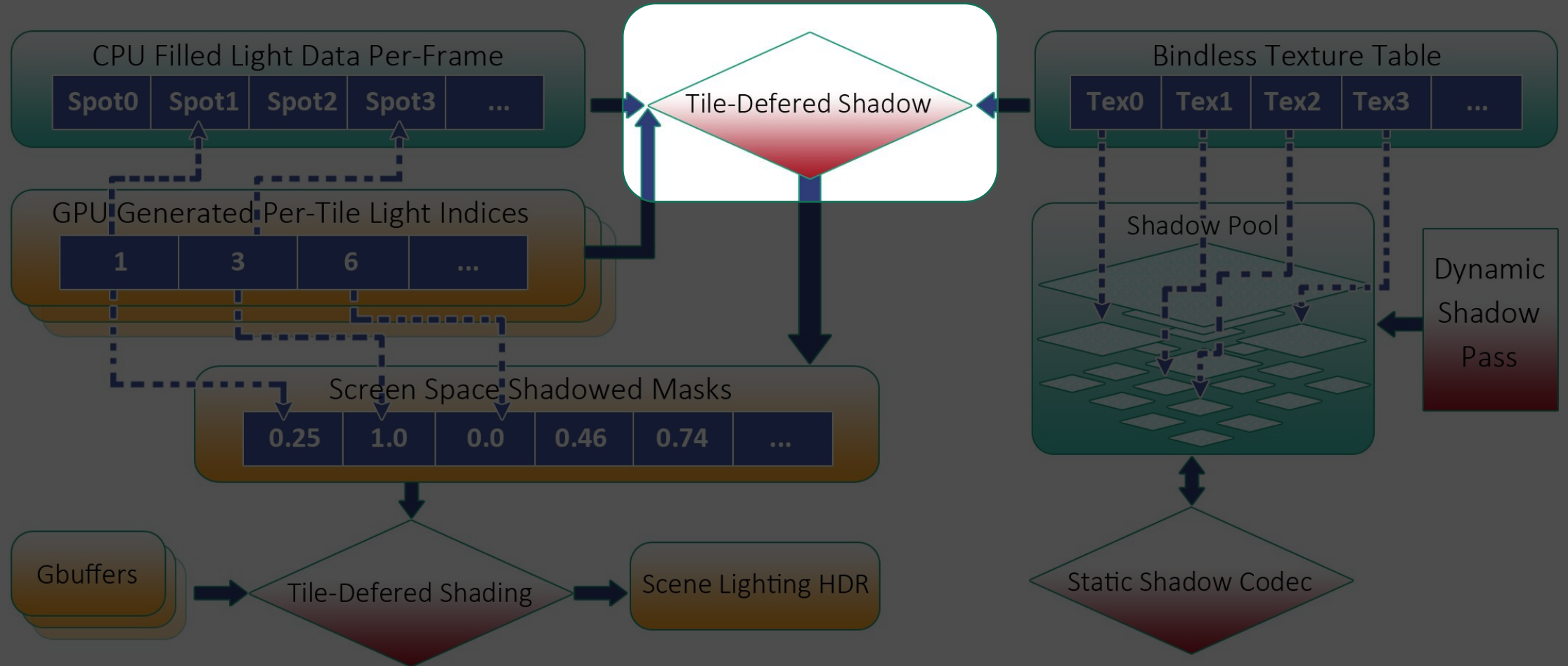


Depth Bias = **1.0 / 65536**

Slope Bias = **FILTER_KERNEL_RADIUS**



Our System





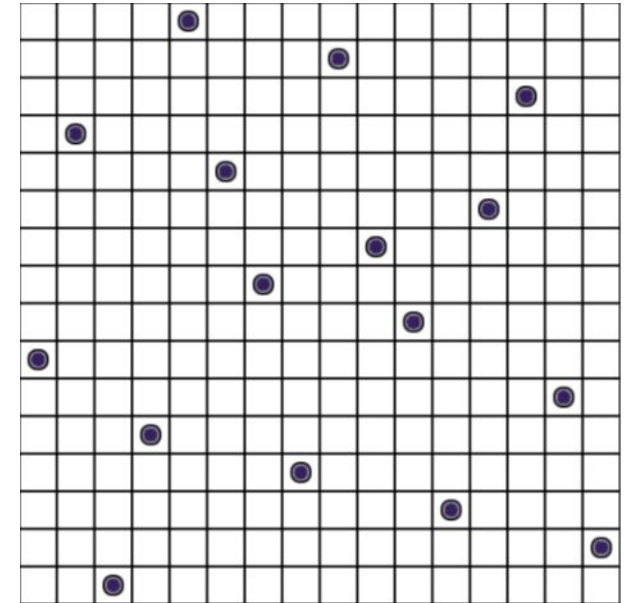
Tiled-Deferred-Shadow

- **Shader Occupancy**
 - Simpler/Small code: Less VGPRs
 - Separate Deferred Spot / Point: Even less VGPRs, Less cache trashing
 - 70% occupancy on GCN
- **Bindless Shadow Map Table: Single Pass Projection**
 - PC: Use DX12 Binding Spaces: Requires SM5.1, Supported on most DX11 GPUs
 - `Texture2D SpotShadows[] : register(t0, space2);`
 - `TextureCube PointShadows[] : register(t0, space3);`
- **DispatchIndirect()** after Light-List Generation



Tiled-Deferred Shadow: Selective Sample Test

- Deferred shadow more sensitive to False-Positives
 - Eating up shadow output channels very quickly
- Fighting **Depth-Complexity**: How Conservative?
 - Near/Far Bounding Boxes + **Selective depth samples test**
 - Cull the light if there's no depth sample touches it
 - Trade off between precision and speed easily



Light List Culling Performance	PS4 Base
2AABB(Bounding Box) Only	0.30ms
2AABB + 16 Depth Sample Test	0.41ms
2AABB + 64 Depth Sample Test	0.57ms



Tiled-Deferred Shadow: Selective Sample Test

- Fighting Depth-Complexity: Comparison



Deferred Lighting Output

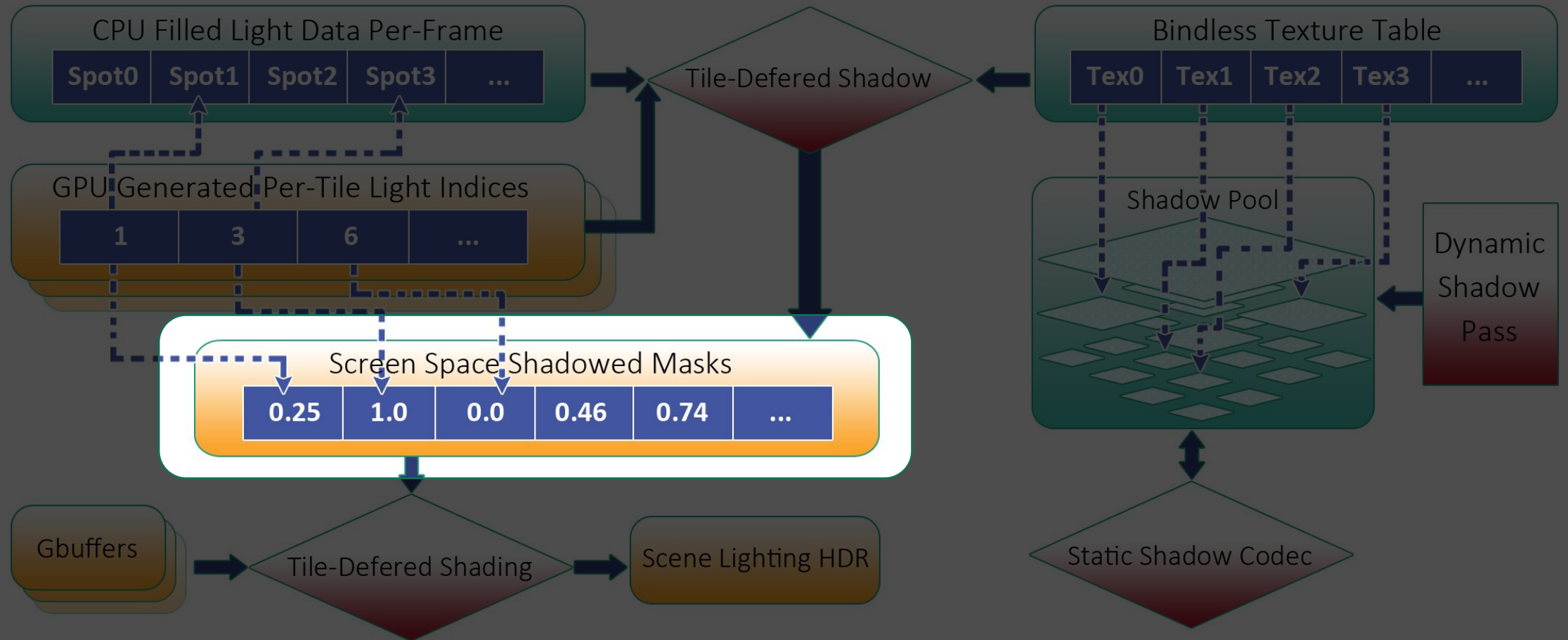


Bounding Box Culling Only



Bounding Box Culling+
Selective Sample Test

Our System





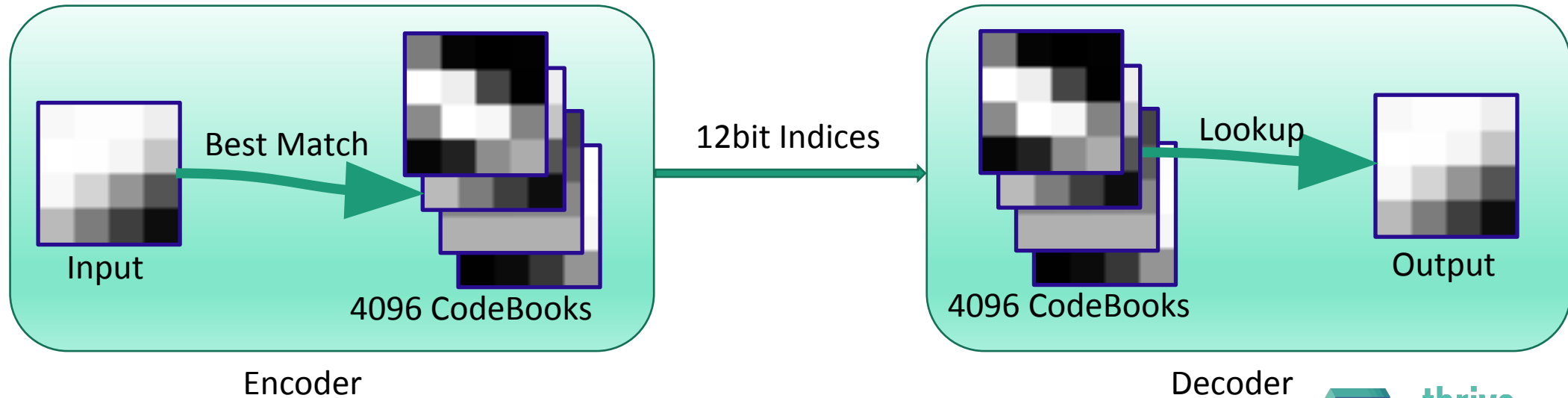
Deferred-Shadow Mask Challenges

- Large Data between Deferred-Shadow -> Deferred-Shading
- Motivation: Targeting 4K+
 - $(4K) * (16 \text{ Shadow Per-Pixel}) * (8 \text{ Bits}) \approx 128 \text{ MB}$
- Better(but naïve) Solution
 - Lower precision masks + Temporal AA
 - As Low as 2 bit is acceptable
 - 128MB -> 32MB
- We can do better than 1bit 😊
 - 128MB -> 12MB



Deferred-Shadow Mask Compression

- Block compression Vector-Quantization(VQ) instead of Pixel Quantization
 - 4x4 Pixel Block, 4096 CodeBooks(Offline-Data-Trained Patterns)
 - Output best matching Indices, 12bits/Block





Deferred-Shadow Mask: Optimization

- Skip fully black/white blocks (WaveBallot)
- **Search Tree**: TSVQ
 - Tree-Structured-Vector-Quantization: $O(\log(n))$
 - Full, Balanced Quad-Tree, 6 levels ($4^6 = 4096$ CodeBooks)
- **MSAD4** (AMD GCN: v_msad_u8)
 - Multimedia instruction
 - Accumulate 4 byte matching errors in one instruction
- **LaneSwizzle** (AMD GCN: ds_swizzle_b32)
 - Fast exchanging data between threads
 - No TGSM (Thread-Group-Shared-Memory)
- Cost: ~7% deferred shadow pass



VQ Compression Code (Per-Pixel Frequency)

```
uint CompressVQ(float Shadow, uint2 Gtid : SV_GroupThreadID, uint GroupIndex : SV_GroupIndex)
{
    uint SrcPixel = uint(Shadow * 254.99f + 1.f) << ((Gtid.x % 4) * 8); //0 is special number for msad
    SrcPixel |= LaneSwizzle(SrcPixel, 0x1F, 0, 0x1);
    SrcPixel |= LaneSwizzle(SrcPixel, 0x1F, 0, 0x2); //Collected 4 neighbor pixels

    uint CurrIndex = -1;
    [unroll]
    for (int i = 0; i < 6; i++) //CodeBook size 4096=4^6
    {
        CurrIndex = CurrIndex * 4 + 4; //QuadTree next level
        uint MatchErr = msad(SrcPixel, uint2(VQCodeBookBuffer[(CurrIndex + Gtid.x % 4) * 4 + (Gtid.y % 4)],
0), 0);

        MatchErr += LaneSwizzle(MatchErr, 0x1F, 0, THREADGROUP_SIZE_X); //Accum next line
        MatchErr += LaneSwizzle(MatchErr, 0x1F, 0, THREADGROUP_SIZE_X << 1); //Accum 2 lines away
        uint MatchErr_Index = (MatchErr << 8) | (Gtid.x % 4); //Pack index for deterministic order
        MatchErr_Index = min(MatchErr_Index, LaneSwizzle(MatchErr_Index, 0x1F, 0, 0x1));
        MatchErr_Index = min(MatchErr_Index, LaneSwizzle(MatchErr_Index, 0x1F, 0, 0x2));
        CurrIndex += MatchErr_Index & 0xf; //Broadcasted best matching of the four children
    }
    return CurrIndex;
}
```



Light Channel: 4 Bits





Light Channel: 2 Bits





Light Channel: 1 Bit





Light Channel: VQ Compressed 0.75 Bit





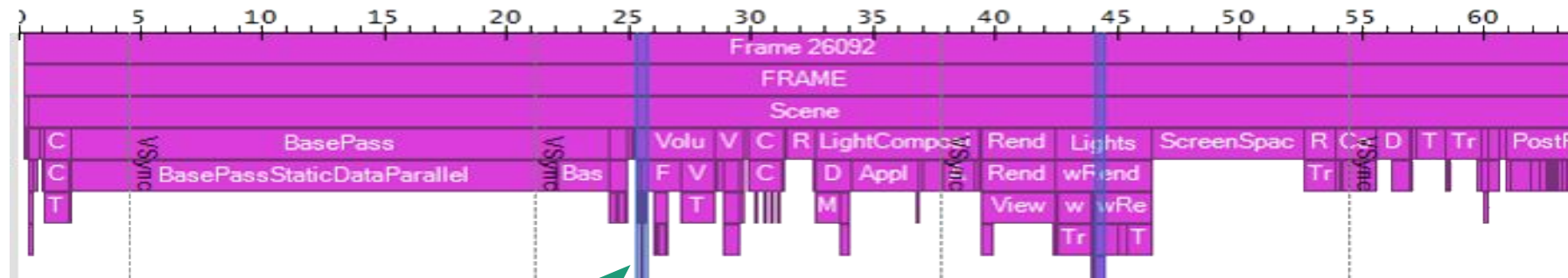
Light Channel: 4 Bits





Performance: Static Camera

- Unannounced Project, running on PS4 Base
 - High-poly un-optimized meshes in BasePass
 - 2507 shadowed lights in the scene

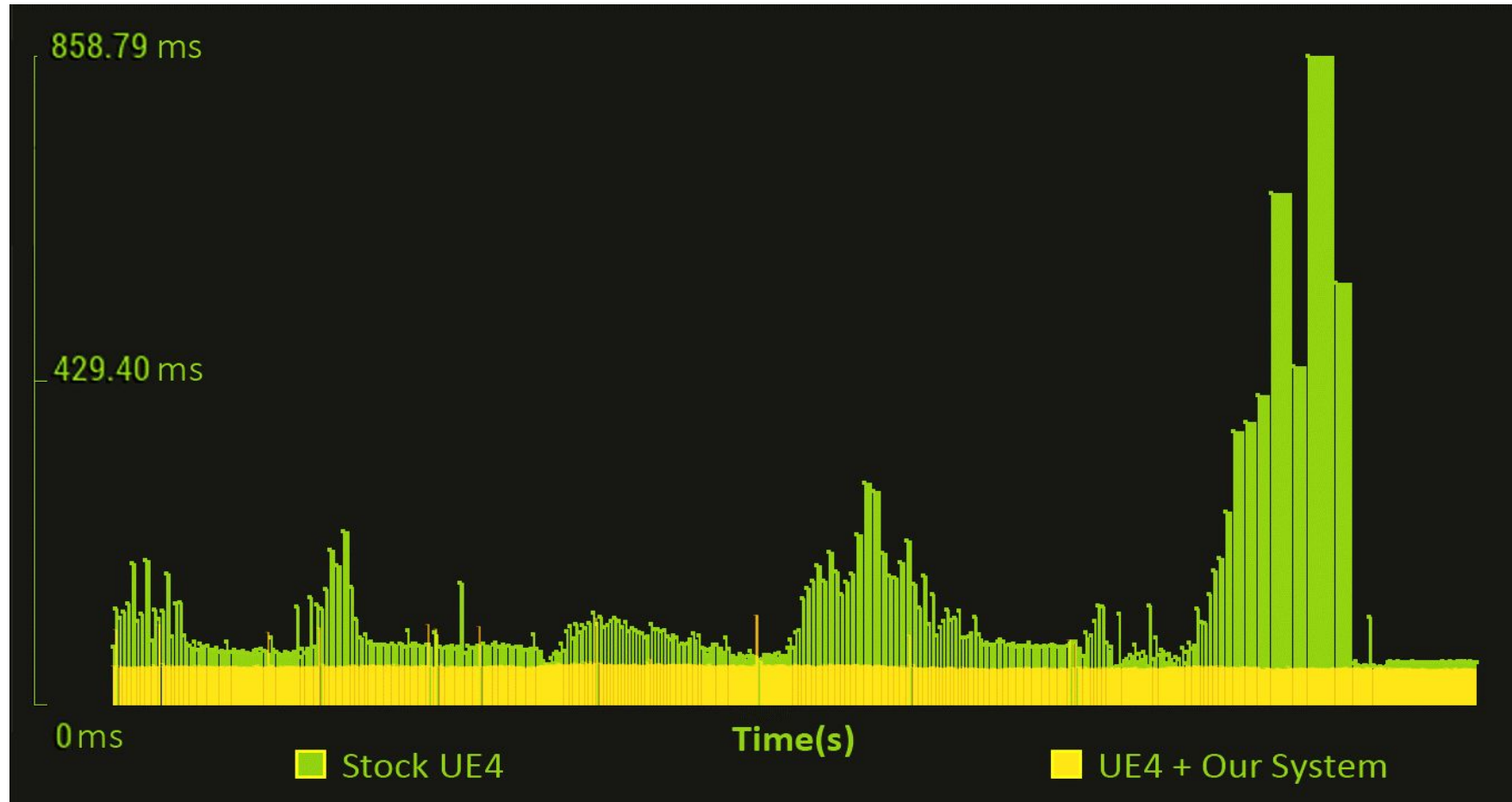


0.44ms Shadow Depth

0.40ms Deferred Shadow



Performance: Moving Camera





Conclusions

- Benefits:
 - Shippable on **Current-Gen** Consoles (PS4/Xbox One/DX12 API)
 - **Thousands** of Shadowed-Light in **Large Environment**
 - Significantly more **Stable** framerate
 - Minimal Shadow-popping
 - Minimized Run-time Memory Allocations
 - Supports Shadowed Volumetric and Transparency Lighting for Local Lights
- Challenges:
 - Vertex-Animated Static-Mesh (E.g. Trees): Static or Dynamic?
 - Currently switched to dynamic with high resolution shadow, otherwise cached
 - Bake stateless texture space animation?



- References

- https://developer.amd.com/wordpress/media/2012/10/AMD_Southern_Islands_Instruction_Set_Architecture.pdf
- <https://gpuopen.com/amd-gcn-assembly-cross-lane-operations/>

- Thanks:

- Zaratsyan, Art
- Béliveau, Jimmy
- Lassonde, Gabriel
- Turcotte, Sebastien
- Fatnassi, Sammy
- Wu, Shan

We're
HIRING



